

ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

NEUMANN JÁNOS
INFORMATIKAI KAR



SZAKDOLGOZAT

OE-NIK	Hallgató neve:	Zöldi Máté Ferenc
2026	Hallgató törzskönyvi száma:	T010171/FI12904/N

Tartalomjegyzék

1. BEVEZETÉS 4
2. PROBLÉMA ELEMZÉSE, SPECIFIKÁCIÓ 5
2.1 Probléma elemzése 5
2.1.1 Wanderlog 5
2.1.2 TripIt 5
2.1.3 Pebblar 5
2.1.4 Sygic Travel 6
2.1.5 Rome2Rio 6
2.1.6 hasonló megoldások összefoglalása 6
2.2 Funkciók 7
2.2.1 Útvonal és programtervezés 7
2.2.2 Mobility as a Service 7
2.2.3 Térképes Megjelenítés 8
2.2.4 Kisvárosok és automatikus POI importálása 8
2.2.5 Valós Együttműködés 9
2.2.6 Költségtervezés 9
2.3 Technológia 9
2.4 Rendszerrel szembeni alapvető követelmények 9
3. LEHETSÉGES MEGKÖZELÉÍTÉSI MÓDOK 10
3.1 útvonal tervezési algoritmusok 10
3.1.1 Dijkstra 10
3.1.2 A*(A-star, A-csillag) 10
3.1.3 Genetikus Algoritmus 11
3.2 Haversine-formula 13
3.3 Borda Count Módszer: 13
3.4 Vizsgált módszerek összevetése: 13
4. VÁLASZTOTT MEGOLDÁSOK ÉS MEGVALÓSÍTÁSOK 14
4.1 Frontend 14
4.2 Backend 14
4.2.1 Greedy távolságalapú módszer (Nearest Neighbor) 15
4.2.2 Greedy prioritásalapú rendezés 16
4.2.3 AI-alapú greedy algoritmus: 16
4.2.4 Automatikus POI-kiválasztás sugár alapján (AI Discover Mode) 16
4.3 Felhasznált külső API-k és szolgáltatások 17
4.3.1 Nominatim API – geokódolás 17
4.3.2 Overpass API – automatikus POI-importálás 17
4.3.3 OpenRouteService – valós útvonalgeometria 17
4.3.4 OpenStreetMap és Leaflet – térképes megjelenítés 18
4.3.5 OpenAI API – AI-alapú POI-kiegészítés 18
4.4 Adatbázis 19

5. RÉSZLETES SPECIFIKÁCIÓ 21
5.1 FELHASZNÁLÓKEZELÉS 21
5.2 LÁTVÁNYOSSÁGOK KEZELÉSE 21
5.3 ÚTVONAL GENERÁLÁSA 22
5.4 ÉRTÉKELÉSI RENDSZER 22
5.5 TÉRKÉPES MEGJELENÍTÉS 23
5.5.1 OPENROUTESERVICE SZEREPE AZ ÚTVONALMEGJELENÍTÉSBEN 23
5.5.2 VALÓS ÚTVONALGEOMETRIA LEKÉRÉSE 24
5.6 AUTOMATIKUS POI-IMPORTÁLÁS 24
5.7 AI PREFERENCIÁK 25
6. RENDSZER BEMUTATÁSA 26
6.1 FUNKCIÓK BEMUTATÁSA: 27
6.2 KEZDŐ FELÜLET (INDEX.HTML) 27
6.3 BEÁLLÍTÁSOK (USER-PREFERENCES.HTML) 28
6.5 ÚJ ÚTVONAL (NEW-ITINERARY.HTML) 30
6.6 TÉRKÉPES MEGJELENÍTÉS (MAP.HTML) 31
6.7 ÉRTÉKELÉSEK (POI-REVIEWS.HTML) 32
6.8 ELŐZMÉNYEK MEGTEKINTÉSE (ITINERARIES.HTML) 33
7. IRODALOMJEGYZÉK 34
8. ÁBRAJEGYZÉK 36

1. BEVEZETÉS

Az utóbbi időszakban többször szerveztem nagyobb külföldi utakat, illetve városnézéseket. Ezek megtervezése gyakran több napot is igénybe vehet, mivel időigényes feladat a különböző utazóblogok áttekintése, a látnivalók kiválasztása, a szállás környezetének vizsgálata, valamint az útvonal térképen történő megtervezése. A cél minden esetben az, hogy a nevezetességeket minél hatékonyabb sorrendben lehessen bejárni, és ne legyen szükség felesleges oda-vissza közlekedésre a városon belül.

A szervezési idő csökkentése érdekében a projekt célja egy olyan rendszer létrehozása, amely az adott helyszín főbb nevezetességeit összegyűjti, majd a felhasználó által megadott preferenciák alapján rangsorolja és útvonalba rendezi. A rendszer figyelembe veszi többek között a bejárás módját, a látványosságok távolságát, prioritását és értékeléseit, majd ezek alapján optimalizált turisztikai útvonalat készít.

A TripPlanner célja egy testreszabott, AI-támogatott útvonaltervező rendszer megvalósítása, amely nemcsak nagyvárosok, hanem kisebb települések esetében is használható. A rendszer külső adatforrások és AI-alapú kiegészítések segítségével képes látványosságokat keresni, azokat értékelni, majd a felhasználói igényekhez igazodó útvonalat előállítani.

2. PROBLÉMA ELEMZÉSE, SPECIFIKÁCIÓ

2.1 Probléma elemzése

Az elképzelt rendszer megtervezése előtt fontosnak tartottam néhány, a piacon már elérhető utazástervező alkalmazás áttekintését. A vizsgálat célja az volt, hogy feltárjam, milyen funkciókat kínálnak a hasonló rendszerek, milyen előnyökkel rendelkeznek, illetve milyen hiányosságok figyelhetők meg bennük. Az elemzés során öt alkalmazást vizsgáltam meg részletesebben: a Wanderlogot, a TripIt-et, a Pebblart, a Sygic Travelt és a Rome2Rio-t. Ezek mind kapcsolódnak az utazástervezéshez, azonban eltérő hangsúlyokkal és eltérő felhasználási célokra készültek.

2.1.1 Wanderlog

A Wanderlog egy átfogó utazástervező alkalmazás, amely több funkciót is biztosít az utazások szervezéséhez. Segítségével a felhasználók nemcsak az útvonalukat tudják megtervezni, hanem az utazáshoz kapcsolódó költségvetést is kezelhetik. Ez különösen hasznos lehet hosszabb vagy több állomásból álló utak esetén, ahol a szállások, programok és közlekedési költségek nyomon követése fontos szempont. Az alkalmazás egyik előnye, hogy több felhasználó is tudja egy időben módosítani az utazási tervet, ami megkönnyíti a csoportos utazások szervezését és gördülékenyebbé teszi a kommunikációt az utazók között. Emellett az offline elérés is jelentős előnyt jelent, mivel így az utazási terv internetkapcsolat nélkül is használható [1].

A Wanderlog hátrányai közé tartozik, hogy nagyobb, összetettebb tervek esetén a működése lassabbá válhat. Emellett a felhasználói visszajelzések alapján előfordulhatnak stabilitási problémák, például váratlan leállások vagy összeomlások. További korlátot jelent, hogy bizonyos adatok megadása kézi adatbevitelt igényel, ami időigényes lehet, különösen akkor, ha a felhasználó sok helyszínt, programot vagy foglalási adatot szeretne rögzíteni. [1]

2.1.2 TripIt

A TripIt elsősorban az automatikus itinerárium-készítésre épít. Az alkalmazás egyik legfontosabb előnye, hogy képes az e-mailben érkező foglalási visszaigazolások alapján automatikusan összeállítani az utazási tervet. Ez jelentősen csökkenti a kézi adatbevitel szükségességét, hiszen a repülőjegyek, szállásfoglalások vagy egyéb utazási dokumentumok adatai automatikusan bekerülhetnek az itineráriumba. Az alkalmazás felülete egyszerűen használható, így azok számára is megfelelő lehet, akik nem szeretnének bonyolult tervezőrendszert használni. A Pro verzió további előnye, hogy valós idejű értesítéseket biztosít, például járatváltozásokkal vagy késésekkel kapcsolatban.

A TripIt hátránya, hogy több hasznos funkció csak fizetős formában érhető el. Ez korlátozhatja az ingyenes verzió használhatóságát, különösen azok számára, akik teljes körű utazásszervező megoldást keresnek. Előfordulhatnak szinkronizációs problémák is, amelyek megnehezíthetik az adatok naprakészen tartását. Emellett a webes felület megjelenése és működése kevésbé modern más, újabb utazástervező alkalmazásokhoz képest.

2.1.3 Pebblar

A Pebblar személyre szabott utazástervezési lehetőségeket kínál, és elsősorban vizuális, térképes útvonaltervezésre alkalmas. Az alkalmazás előnye, hogy támogatja több felhasználó bevonását a tervezési folyamatba, ezért csoportos utazások esetén is jól használható. A felhasználók közösen alakíthatják ki az útvonalat, módosíthatják a programokat, valamint összehangolhatják az utazás részleteit. Az offline elérés szintén hasznos funkció, mivel az elkészült tervek internetkapcsolat nélkül is hozzáférhetők lehetnek.

Hátrányként említhető, hogy a Pebblar több esetben kézi adatbevitelt igényel. Nem szinkronizálja automatikusan az e-mail-fiókba érkező szállás-, repülő- vagy egyéb foglalási adatokat, ezért ezeket a felhasználónak külön kell rögzítenie. Az alkalmazás fizetős modellje szintén korlátozhatja a használatát. További probléma lehet, hogy az adatok frissessége nem minden esetben garantált, így előfordulhat, hogy a megjelenített információk nem teljesen felelnek meg a valóságnak. [2]

2.1.4 Sygic Travel

A Sygic Travel egyik legnagyobb előnye az offline térképes működés. Az alkalmazás lehetővé teszi, hogy a felhasználók internetkapcsolat nélkül is hozzáférjenek a térképekhez és az útvonaltervekhez. Ez különösen külföldi utazások során lehet hasznos, amikor a mobilinternet használata korlátozott vagy költséges. Az alkalmazás automatikusan javasol látványosságokat különböző kategóriák alapján, ami megkönnyíti az utazás programjának összeállítását. További érdekessége, hogy bizonyos helyszínekhez 360 fokos fotókat is biztosít, amelyek segítenek a látványosságok előzetes megismerésében.

A Sygic Travel korlátja, hogy nem biztosít valódi, AI-alapú, felhasználói preferenciákon alapuló útvonaloptimalizálást. Bár a rendszer képes látványosságokat ajánlani, a tervezés kevésbé testre szabható, és nem veszi figyelembe mélyebb szinten az egyéni utazási szokásokat vagy súlyozott preferenciákat. A csoportos együttműködés lehetősége is korlátozott, ezért kevésbé alkalmas olyan utazások szervezésére, ahol több felhasználó közösen szeretné alakítani a programot. [3]

2.1.5 Rome2Rio

A Rome2Rio elsősorban multimodális útvonaltervezőként ismert. Az alkalmazás nagy előnye, hogy több közlekedési módot is figyelembe vesz, például repülőt, vonatot, buszt, hajót, taxit vagy autós útvonalakat. Globális lefedettsége miatt sok országban és városban használható, és gyorsan képes különböző útvonal-opciókat megjeleníteni. A felülete könnyen kezelhető, a keresések gyorsak, ezért különösen hasznos lehet akkor, ha a felhasználó két helyszín között szeretne közlekedési lehetőségeket összehasonlítani

A Rome2Rio ugyanakkor nem klasszikus értelemben vett, személyre szabott utazástervező alkalmazás. Nem készít részletes, felhasználói preferenciák alapján optimalizált útvonaltervet, és nem támogatja a látványosságok bejárási sorrendjének optimalizálását. Emellett nem kínál komplex költségtervezési vagy értékelési funkciókat sem. Emiatt inkább közlekedési útvonalak keresésére alkalmas, mintsem teljes utazási programok [4]

2.1.6 hasonló megoldások összefoglalása

Ezek a rendszerek jól mutatják, hogy az utazástervező alkalmazások esetében kulcsfontosságú a felhasználóbarát felület, a rugalmas tervezés, az offline vagy térképes támogatás, valamint a személyre szabhatóság. A vizsgált megoldások több hasznos funkcióval rendelkeznek, azonban nem minden esetben támogatják megfelelően a kisebb települések turisztikai feldolgozását, a felhasználói preferenciákon alapuló útvonaloptimalizálást, illetve a látványosságok értékeléseinek beépítését az ajánlási folyamatba.

A TripPlanner ezekre a hiányosságokra kínál megoldást. A rendszer AI-támogatott módon képes látványosságokat keresni, azokat prioritás, értékelés és távolság alapján rangsorolni, majd a felhasználói preferenciák figyelembevételével optimalizált útvonalat készíteni. A projekt egyik fő innovációja, hogy nem kizárólag nagyvárosokra koncentrál, hanem automatikus POI-importálás segítségével kisebb települések esetében is használható útvonaltervezést biztosít.

2.2 Funkciók

A TripPlanner rendszer több egymásra épülő funkcióból áll. A felhasználó megadhatja az indulási helyet, amelyet a rendszer koordinátává alakít. Ezután a rendszer képes környékbeli látványosságokat keresni, azokat pontozni, majd útvonalat generálni belőlük. Az útvonaltervezés során figyelembe veszi a távolságot, a POI-k prioritását, a felhasználói értékeléseket és a felhasználó által megadott preferenciákat.

2.2.1 Útvonal és programtervezés

Egy utazástervező rendszer egyik legfontosabb feladata, hogy a felhasználó számára optimális útvonalat állítson össze az adott helyszín látványosságaiból. Ehhez első lépésben szükség van a környék főbb nevezetességeinek összegyűjtésére, majd ezek rangsorolására a felhasználói igények és a rendelkezésre álló adatok alapján.

A TripPlanner rendszer a felhasználó által megadott kiindulási helyből indul ki. A felhasználó nem koordinátákat ad meg, hanem egy címet vagy helyszínnevet, amelyet a rendszer a Nominatim API segítségével szélességi és hosszúsági koordinátává alakít. A koordináták alapján a rendszer képes meghatározni, hogy mely POI-k találhatók a megadott környezetben.

A látványosságok kiválasztásánál és rangsorolásánál több tényező is szerepet kap. A rendszer figyelembe veszi a POI-k távolságát, prioritását, felhasználói értékeléseit, valamint a felhasználó által beállított preferenciákat. Ilyen preferencia lehet például, hogy a felhasználó minél kevesebbet szeretne sétálni, inkább a magasabb értékelésű helyeket szeretné előnyben részesíteni, vagy több látnivalót szeretne bejárni egy útvonalon belül.

A TripPlanner két fő útvonaltervezési módot biztosít: kézi és automatikus útvonaltervezést. Kézi útvonaltervezés esetén a felhasználó maga választja ki a megjelenített POI-listából azokat a látványosságokat, amelyeket szeretne meglátogatni. A kiválasztott pontok sorrendjét a rendszer genetikus algoritmus segítségével optimalizálja. Ez a módszer több lehetséges bejárási sorrendet vizsgál, majd ezek közül választja ki a legkedvezőbb útvonalat.

Automatikus útvonaltervezés esetén a felhasználónak nem kell kézzel kiválasztania a látványosságokat. Ebben a módban a felhasználó megadja a kiindulási pontot, a keresési sugarat és a maximálisan bevonni kívánt POI-k számát. A rendszer ezután a megadott körzetben található helyszíneket összegyűjti, pontozza, majd a távolság, prioritás, értékelések és felhasználói preferenciák alapján kiválasztja a leginkább ajánlott látványosságokat.

Az útvonaltervezés során a rendszer a közlekedési módot is kezeli. A felhasználó választhat például gyalogos és autós közlekedés között. Ez a választás a térképes megjelenítésnél is szerepet kap, mivel a rendszer a kiválasztott közlekedési mód alapján OpenRouteService segítségével valós útvonalgeometriát kérhet le.

A rendszer által generált útvonalak mentésre kerülnek az adatbázisba, így a felhasználó később is visszakeresheti és újra megnyithatja őket. Ez lehetővé teszi, hogy az alkalmazás ne csak egyszeri útvonaltervezésre, hanem korábbi utazási tervek kezelésére is alkalmas legyen.

2.2.2 Mobility as a Service

A Mobility as a Service egy modern közlekedési paradigma, aminek célja, hogy a különböző közlekedési módokat egyetlen integrált platformon keresztül személyre szabottan és digitálisan elérhetővé tegye a felhasználó számára.

A különböző közlekedési módok összehangolásával hatékonyabb útvonalakat tudunk tervezni. A projektben a Mesterséges Intelligencia alkalmazásával támogatjuk a dinamikus választást olyan

módon, hogy értékeljük az útvonalakat. Az értékeléseknél figyelembe lehet venni az utazás idejét a költségvetésünket vagy akár, hogy milyen környezeti hatást fejt ki az utazásunk.

A jelenlegi rendszerben a MaaS szemlélet egyszerűsített formában jelenik meg: a felhasználó választhat közlekedési módot, például gyalogos vagy autós útvonalat. Ez később bővíthető tömegközlekedési, kerékpáros vagy több közlekedési módot kombináló útvonaltervezéssel.

2.2.3 Térképes Megjelenítés

A rendszer térképes megjelenítése Leaflet.js segítségével történik. A létrehozott útvonalak térképen is megtekinthetők, ahol a kiindulási pont, az egyes POI-k és az útvonal vonala is megjelenik.

Az útvonaladatokat a backend GeoJSON formátumban adja vissza. A GeoJSON tartalmazza a startpontot, az útvonalba rendezett látványosságokat, valamint az útvonal geometriai adatait. A POI-k sorszámozva jelennek meg a térképen, így a felhasználó könnyen követheti a javasolt bejárási sorrendet.

A térképes megjelenítés OpenRouteService integrációval is rendelkezik. Ennek segítségével a rendszer képes valós úthálózaton alapuló útvonalgeometriát lekérni. A felhasználó által választott közlekedési mód alapján a rendszer gyalogos vagy autós útvonalat jeleníthet meg.

Amennyiben a külső útvonaltervező szolgáltatás nem érhető el, a rendszer továbbra is képes légvonalbeli útvonalat rajzolni. Ez biztosítja, hogy a térképes megjelenítés akkor is működőképes maradjon, ha a külső API nem ad vissza érvényes útvonalat.

A térképen megjelenő POI-khoz popup ablakok is tartoznak. Ezekben megjelenhet a látványosság neve, leírása, értékelései, valamint egy hivatkozás az értékelési oldalra.

2.2.4 Kisvárosok és automatikus POI importálása

A legtöbb ma elérhető alkalmazás a nagyobb városokra fókuszál. Ezért hiánypótlóként szolgálhat, ha egy kisebb vidéki városnak is ki tudjuk gyűjteni a nevezetességeit. Szerencsére a Mesterséges intelligencia egy utasítással képes akár karakószörcsög és a környékén található látványosságokat is kigyűjteni.

A rendszer egyik fontos funkciója az automatikus POI-importálás. Ennek célja, hogy az alkalmazás ne kizárólag előre rögzített látványosságokkal dolgozzon, hanem képes legyen a felhasználó által megadott hely környezetében új helyszíneket is keresni.

Az automatikus importálás az Overpass API segítségével történik, amely az OpenStreetMap adatbázisából kér le turisztikai és történelmi jelentőségű helyszíneket. A rendszer például múzeumokat, kilátópontokat, emlékműveket, galériákat, várakat, romokat és egyéb látványosságokat tud keresni a megadott koordináta környezetében.

A lekért helyszíneket a rendszer feldolgozza, majd összeveti a már meglévő adatbázisban szereplő POI-kkal. Ez megakadályozza, hogy ugyanaz a látványosság többször is bekerüljön a rendszerbe. Az újonnan talált helyszínek eltárolásra kerülnek, így később ugyanúgy felhasználhatók útvonaltervezéshez, mint a korábban rögzített POI-k.

Ez a megoldás lehetővé teszi, hogy az alkalmazás kisebb településeken vagy kevésbé ismert turisztikai helyszíneken is használható legyen, ahol nem feltétlenül áll rendelkezésre előre összeállított látványosság-adatbázis.

2.2.5 Valós Együttműködés

A felhasználók általában nem utaznak egyedül ezért érdemes egy olyan funkcióval bővíteni az alkalmazást, amivel egy kisebb társaság minden tagja egyénileg beleszólhat a tervezésbe.

2.2.6 Költségtervezés

Egy utazás tervezésekor érdemes feljegyezni milyen költségekkel számolhatunk ezt az adott nevezetességek jegyáraiból, az előre lefoglalt szállás, repülőjegy árából ki lehet számolni. Érdemes erre a funkcióra egy külön felületet biztosítani az alkalmazáson belül

2.3 Technológia

A .NET keretrendszer stabil alapot nyújt a projekt számára, különösen a Windows-alapú környezetekben. A fő erősségek a skálázhatóság, a biztonság és a többnyelvű támogatás (C#, F#, VB.NET). A kihívások közé tartozik a platformfüggőség (pl. Linux támogatás hiánya), amelyet a jövőben .NET Core-ra való migrációval lehet kezelni.

2.4 Rendszerrel szembeni alapvető követelmények

A TripPlanner rendszer célja, hogy a felhasználók számára olyan útvonaltervezési megoldást biztosítson, amely több turisztikai látványosság figyelembevételével képes útvonaljavaslatot készíteni. A rendszerrel szembeni alapvető elvárás, hogy a felhasználó ne koordinátákat adjon meg, hanem természetes formában, például cím vagy helyszínnév alapján indíthassa el a tervezést. A megadott helyszínből a rendszernek útvonaltervezésre alkalmas földrajzi adatokat kell előállítania.

A rendszernek kezelnie kell a látványosságokat, azok földrajzi elhelyezkedését, leírását, prioritását és értékeléseit. Emellett szükséges a felhasználókhoz kapcsolódó adatok és preferenciák tárolása is, mivel az útvonaltervezés során nem minden felhasználó számára ugyanaz a sorrend vagy útvonal tekinthető optimálisnak. Egyes felhasználók számára a rövidebb távolság, mások számára a magasabb értékelésű vagy nagyobb prioritású látványosságok lehetnek fontosabbak.

A rendszerrel szembeni további követelmény, hogy támogassa az automatikus POI-keresést és -importálást. Ez különösen kisebb települések esetében fontos, ahol nem feltétlenül áll rendelkezésre előre összeállított turisztikai adatbázis. A rendszernek képesnek kell lennie arra, hogy a felhasználó által megadott kiindulási pont környezetében látványosságokat keressen, azokat feldolgozza, majd útvonaltervezéshez felhasználható formában eltárolja.

Az útvonaltervezés során a rendszernek figyelembe kell vennie a távolságot, a látványosságok prioritását, az értékeléseket és a felhasználói preferenciákat. Az elkészült útvonalakat térképen is meg kell jeleníteni, hogy a felhasználó könnyen értelmezhesse a javasolt bejárási sorrendet. A rendszernek továbbá biztosítania kell a korábban létrehozott útvonalak mentését és későbbi visszakeresését.

3. LEHETSÉGES MEGKÖZELÉÍTÉSI MÓDOK

3.1 útvonal tervezési algoritmusok

Az útvonaltervezési problémák megoldására több különböző algoritmikus megközelítés alkalmazható. A választott módszer függ attól, hogy a feladat egyetlen kiindulási és célpont közötti legrövidebb út meghatározása, vagy több látványosság optimális sorrendjének kialakítása. A TripPlanner szempontjából mindkét megközelítés releváns: egyrészt szükség van a távolságok becslésére, másrészt több POI bejárási sorrendjének meghatározására.

3.1.1 Dijkstra

[5]Leginkább elterjedt útvonalkereső algoritmus. Edsger W. Dijsktraról elnevezett módszer. A problémát a következőképpen fogalmazta meg [6]:

Adott egy *n* pontból álló gráf, ahol az élekhez súly (távolság vagy költség) van rendelve. A cél: megtalálni az összes csúcsot összekötő, legkisebb összköltségű fát – azaz minden csúcsból egyértelmű út vezessen minden másikhoz.

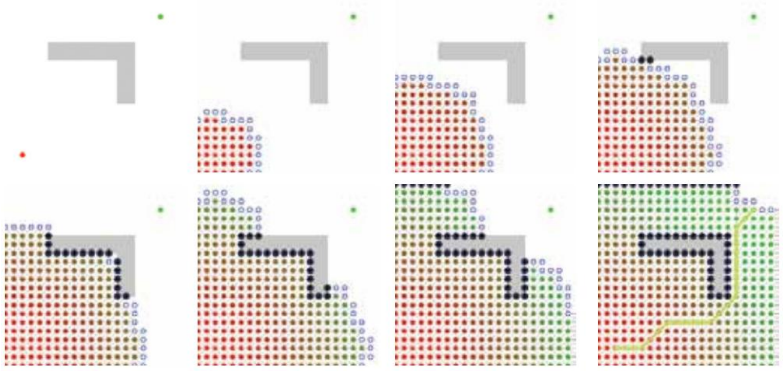
A módszer három élhalmazzal dolgozik. Az elfogadott élek,a következő vizsgált élek, még nem vizsgált élek

A csúcsokat két részre osztjuk az összekapcsolt csúcsokra és a még be nem kapcsolt csúcsokba.

Az algoritmus menete:

1. A II. halmazból kiválasztjuk a legrövidebb élt, és hozzáadjuk az I.-hez, a kapcsolódó B csúcsot áthelyezzük A-ba.
2. Az új A-beli csúcshoz tartozó éleket vizsgáljuk: ha rövidebbek, mint a meglévő II.-beliek, cseréljük őket.

Ezt a két lépést addig ismételjük, amíg minden csúcs bekerül az A halmazba. Az így kapott I. halmaz adja a legkisebb költségű összekötő fát. [7]



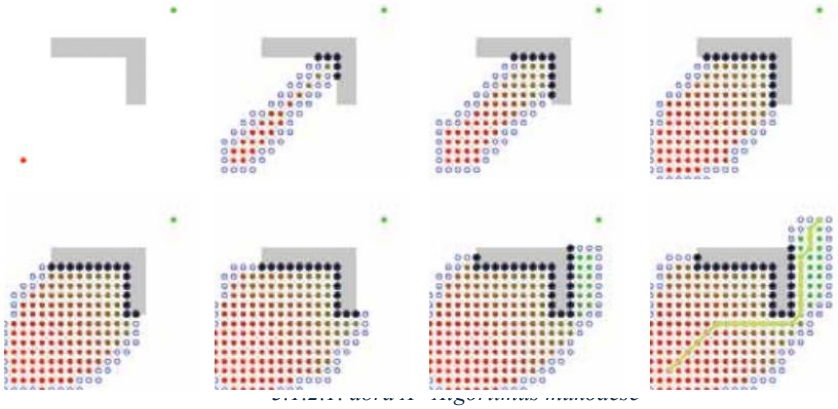
3.1.1.1. ábra Dijkstra Algoritmus Működése

3.1.2 A*(A-star, A-csillag)

Az A* algoritmust Bertram Raphael, Nils Nilsson és Peter Hart alkotta meg 1968-ban. Az algoritmus a legjobbat-először keresést valósítja meg, súlyozva a csúcsokat egy kiértékelő

függvény f(n) segítségével. Ennek kulcseleme a heurisztikus függvény h(n), amely az n csomóponttól a célig vezető legolcsóbb út becsült útköltségét adja meg, amely légvonalbeli távolság alapján számítódik. [7]

Az algoritmus a következőképpen működik: a keresés kezdetekor rendelkezésre áll a keresési gráf, amelyben adott a kezdő és célpont. A h(n) heurisztikus függvény rendelkezésre áll minden n csúcshoz, és ismertek a szomszédos csúcsok távolságai is. A működéshez szükséges egy, a már megvizsgált elemeket (jelölje: Z) és a már felderített, de be nem járt csúcsokat (jelölje: Y) tartalmazó lista. Itt tárolja az eljárás azon pontokat, amelyeket felderített és megvizsgált, illetve a még be nem járt, de már ismert pontokat. A hatékonyabb és gyorsabb lefutás érdekében a csak felderített csomópontokat az f(n) szerint sorba rendezve érdemes tárolni, így a legkedvezőbb csomópont már rendelkezésre áll [8]



3.1.3 Genetikus Algoritmus

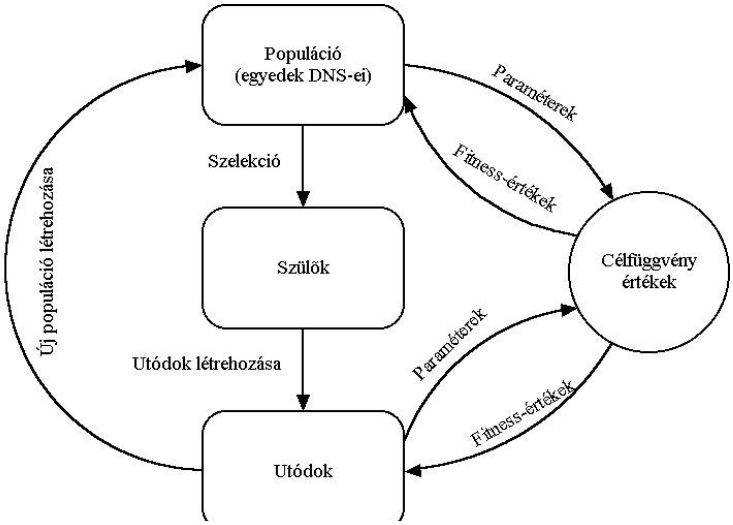
A Genetikus Algoritmus célja, hogy megtalálja a legoptimálisabb útvonalat miközben a felhasználó időt takarít meg.

A genetikus algoritmus nem vizsgálja meg az összes lehetséges útvonalat, hanem több kezdeti útvonalváltozatból indul ki, majd ezek iteratív javításával keres kedvező megoldást.. A fitness függvény alkalmazással kiértékeljük az útvonalakat figyelembe véve az utazási időt, a helyszínek számát és az útvonal gazdaságosságát. A Hatékony útvonalak nagy eséllyel kerülnek a következő generációba

A következő lépésben a jobb teljesítményt nyújtó útvonalakat kiválasztjuk majd örökítjük a jellemzőiket. A keresztezés során a kiválasztott útvonalat kombináljuk, hogy egy potenciálisan jobb útvonalat hozhassunk létre. Majd sorrendkeresztezés alkalmazással megőrizzük a régi egyedet miközben kombináljuk az új útvonallal. Ezt követően véletlenszerűen megváltoztatunk egyes elemeket (pl a látványosságok sorrendjét) így növelve a megoldás változatosságát és elősegítve az algoritmust, hogy ne ragadjon be a lokális optimumhoz

Ezek a folyamatok addig ismétlődnek meg míg egy előre meg nem határozott számú generációt le nem futtat vagy amíg a megoldás nem javul már érdemben tovább.

A megoldás lehetővé teszi a z útvonalak hatékonyágát más módszerekhez képest lehetővé téve, hogy az utazók rövidebb idő alatt több látványosságot nézzenek meg



3.1.3.1. ábra Genetikus Algoritmus működése

A TripPlanner rendszer a látványosságok optimális sorrendjének meghatározására genetikus algoritmust alkalmaz.

A genetikus algoritmus célja, hogy a kiválasztott POI-kból olyan sorrendet hozzon létre, amely a teljes bejárási útvonal hosszát csökkenti. Ez különösen fontos turisztikai útvonaltervezésnél, ahol nem csupán egyetlen célpont elérése a feladat, hanem több látványosság hatékony sorrendbe rendezése.

Az algoritmus első lépésként több véletlenszerű útvonalváltozatot hoz létre. Ezek az útvonalak alkotják a kezdeti populációt. Egy útvonal egy lehetséges POI-sorrendet jelent, például azt, hogy a felhasználó milyen sorrendben látogassa meg a kiválasztott helyszíneket.

Ezután a rendszer minden útvonalhoz kiszámít egy fitness értéket. A fitness érték azt mutatja meg, hogy az adott útvonal mennyire kedvező. A jelenlegi működésben ennek alapja a teljes útvonalhossz: minél rövidebb a bejárási távolság, annál jobb az adott megoldás.

A jobb fitness értékkel rendelkező útvonalak nagyobb eséllyel kerülnek kiválasztásra a következő generáció létrehozásához. A kiválasztott útvonalakból keresztezés segítségével új útvonalváltozatok jönnek létre. A keresztezés során két útvonal tulajdonságai kombinálódnak úgy, hogy az új útvonal is érvényes POI-sorrendet tartalmazzon.

A változatosság fenntartása érdekében az algoritmus mutációt is alkalmaz. Mutáció során kisebb véletlenszerű módosítás történik, például két látványosság helyet cserél az útvonalban. Ez segít abban, hogy az algoritmus ne ragadjon be egy lokálisan jónak tűnő, de nem feltétlenül legjobb megoldásba.

A folyamat több generáción keresztül ismétlődik. Minden generációban újabb útvonalváltozatok jönnek létre, majd a rendszer ezek közül választja ki a legjobbakat. A folyamat végén az algoritmus a legkedvezőbbnek talált POI-sorrendet adja vissza, amely alapján az útvonal létrejön.

A genetikus algoritmus előnye, hogy nem csak egyetlen útvonalat vizsgál meg, hanem párhuzamosan több lehetséges megoldással dolgozik. Bár nem garantálja minden esetben a globálisan optimális megoldást, a gyakorlatban jó eredményt adhat olyan problémáknál, ahol több látványosság sorrendjét kell optimalizálni.
. [9]

3.2 Haversine-formula

A Haversine-formula két földrajzi koordináta közötti légvonalbeli távolság kiszámítására szolgál. A módszer figyelembe veszi a Föld görbületét, ezért alkalmas szélességi és hosszúsági koordináták közötti távolság becslésére. Útvonaltervező rendszerekben gyakran használható gyors távolságbecslésre, például környékbeli helyszínek szűrésére vagy útvonalváltozatok előzetes kiértékelésére.

Előnye, hogy gyorsan számítható, és nem igényel külső szolgáltatást. Hátránya, hogy nem veszi figyelembe a tényleges úthálózatot, az utcákat, gyalogutakat, forgalmi korlátozásokat vagy természetes akadályokat. Emiatt pontos térképes útvonalmegjelenítéshez célszerű külső útvonaltervező szolgáltatást is alkalmazni.

3.3 Borda Count Módszer:

A Borda Count egy többszempontú rangsorolási módszer, amely akkor használható, ha több alternatívát több különböző szempont alapján kell értékelni. A módszer lényege, hogy minden szempont rangsorolja az alternatívákat, majd az egyes helyezésekhez pontszámokat rendelünk. Az az alternatíva kapja a legjobb összesített eredményt, amely a legtöbb pontot gyűjti össze.

Útvonaltervezés esetén ilyen szempont lehet például a gyorsaság, a kényelem, a fenntarthatóság vagy a költség. A módszer későbbi fejlesztések során alkalmas lehet több alternatív útvonal összehasonlítására. A jelenlegi pilot rendszerben ez inkább továbbfejlesztési lehetőségként jelenik meg, mivel a rendszer fő célja a POI-k rangsorolása és a bejárási sorrend meghatározása. [10]

3.4 Vizsgált módszerek összevetése:

A bemutatott módszerek különböző útvonaltervezési problémákra adnak megoldást. A Dijkstra és az A* algoritmus elsősorban egy kiindulási és egy célpont közötti legrövidebb út meghatározására alkalmas, különösen akkor, ha rendelkezésre áll egy részletes úthálózati gráf. A TripPlanner esetében azonban a fő feladat nem csupán két pont közötti útvonal keresése, hanem több látványosság bejárási sorrendjének meghatározása.

Ezért a projekt szempontjából a genetikus algoritmus jobban illeszkedik a megoldandó problémához, mivel több POI sorrendjének optimalizálására is alkalmazható. A Haversine-formula a gyors távolságszámítást támogatja, míg a valós úthálózaton alapuló útvonalmegjelenítéshez külső szolgáltatás használata indokolt. A Borda Count módszer a későbbi fejlesztések során több útvonalalternatíva összehasonlítására lehet alkalmas.

4. VÁLASZTOTT MEGOLDÁSOK ÉS MEGVALÓSÍTÁSOK

A TripPlanner rendszer célja egy olyan, felhasználóbarát turisztikai útvonaltervező alkalmazás létrehozása, amely képes a felhasználó által megadott kiindulási pont környezetében található látványosságok összegyűjtésére, kiértékelésére, majd ezekből egy optimális sétaútvonal összeállítására. A rendszer jelenlegi állapotában tartalmazza a fő útvonaltervezési, POI-kezelési, értékelési, preferenciakezelési és térképes megjelenítési funkciókat. A rendszer képes a felhasználó által megadott kiindulási pont alapján látványosságokat keresni, azokat rangsorolni, útvonalat generálni, majd az eredményt térképen megjeleníteni.

4.1 Frontend

A rendszer jelenlegi frontendje egy egyszerű, HTML és JavaScript alapú többoldalas felület, amely közvetlenül kommunikál az ASP.NET Core backend API-jaival. A rendszer felépítése moduláris: minden oldal saját funkcionalitással rendelkezik, míg a közös elemek megosztott JavaScript fájlokban kaptak helyet. A frontend működését a config.js határozza meg, amelyben a backend API elérési útvonalát (apiBase), valamint a böngészőben tárolt felhasználói azonosítók kulcsait definiáljuk. A belépett felhasználó azonosítóját és nevét a rendszer localStorage-ben tárolja, amely lehetővé teszi, hogy a különböző oldalak egységesen kezeljék a felhasználói állapotot.

A frontend több különálló HTML-oldalból áll. Az index.html a kezdőoldal és az egyszerű felhasználói belépés felülete, a new-itinerary.html az útvonal létrehozására szolgál, a map.html a térképes megjelenítést biztosítja, az itineraries.html a korábbi útvonalakat listázza, a user-preferences.html az AI-beállítások kezelésére szolgál, míg a poi-reviews.html az egyes látványosságok értékeléseit jeleníti meg.

A felhasználói felület logikáját többségében JavaScript biztosítja: ez végzi az API-hívásokat, frissíti az adatokat, valamint vezérli a felhasználói interakciókat. A térképes megjelenítés a Leaflet.js könyvtárra épül, amely egy könnyen testre szabható, nyílt forráskódú webes térképmotor. A Leaflet lehetővé teszi a POI-k dinamikus megjelenítését, a felhasználó által létrehozott útvonalak kirajzolását GeoJSON formátumból, valamint interaktív elemek hozzáadását.

A frontend fejlesztése során a fókusz a gyors prototípuson volt, ennek megfelelően a jelenlegi implementáció egy könnyen érthető és jól bővíthető struktúrát követ. A hosszú távú cél a Blazor és .NET MAUI alapú egységes felhasználói felület kialakítása, amely lehetővé teszi, hogy a projekt weben és mobilplatformokon is ugyanazzal a C#-alapú kódbázissal működjön. A Blazor Hybrid megközelítés különösen előnyös a projekt jellegéből adódóan, mivel az API-hívások, az AI-funkciók integrációja és a komponensalapú felépítés egységes logikát használhat a teljes alkalmazáson belül. A jelenlegi HTML–JavaScript prototípus azonban kiforrott alapot biztosít a működő fő funkciók bemutatásához és az API-k teszteléséhez.

4.2 Backend

A TripPlanner alkalmazás backendje ASP.NET Core Web API alapokon épül fel, amely modern, gyors és jól bővíthető infrastruktúrát biztosít az útvonaltervezési logika, az adatok kezelése és a felhasználói műveletek kiszolgálására. A backend feladata a frontendről érkező kérések feldolgozása, az adatok mentése és lekérdezése, az útvonaltervezési algoritmusok futtatása, valamint a külső szolgáltatások integrálása.

A rendszer moduláris architektúrával rendelkezik. Az egyes funkciók külön vezérlőkben és kiszolgáló osztályokban kaptak helyet, így a felhasználókezelés, a POI-kezelés, az útvonalgenerálás, az értékelések és a preferenciák kezelése egymástól elkülönülten működik. Ez átláthatóbbá teszi a kódot, és megkönnyíti a későbbi bővítéseket is.

Az adatok kezelését Entity Framework Core biztosítja. Az EF Core gondoskodik az objektum-relációs leképezésről, így a C# osztályokhoz tartozó adatok egyszerűen tárolhatók és visszatölthetők az adatbázisból. A rendszer adatbázisában a felhasználók, a POI-k, az értékelések, az elkészült útvonalak és a felhasználói preferenciák adatai kerülnek eltárolásra.

A backend egyik legfontosabb feladata a látványosságokból álló útvonalak generálása és optimalizálása. A rendszer figyelembe veszi a felhasználó által megadott kiindulási pontot, a kiválasztott vagy automatikusan ajánlott POI-kat, a közlekedési módot, valamint a felhasználói preferenciákat. Az útvonaltervezés során a távolság, a POI-k prioritása és az értékelések is szerepet kapnak.

A működés egyik központi eleme az AI-alapú pontozási logika, amely minden jelölt POI-hoz súlyozott pontszámot rendel. A pontszámítás során a rendszer figyelembe veszi a távolságot, a POI prioritását, az átlagos felhasználói értékelést, valamint a felhasználó által beállított súlyokat. Ennek eredményeként a rendszer nem minden felhasználónak ugyanazt az útvonalat ajánlja, hanem a megadott preferenciák alapján személyre szabott javaslatot készít.

A backend külső szolgáltatásokat is használ. A címek koordinátává alakításához geokódolás történik, az automatikus POI-importáláshoz Overpass API kerül alkalmazásra, a valós útvonalgeometria lekéréséhez pedig OpenRouteService integrációt használ a rendszer. Ezek segítségével az alkalmazás nemcsak statikus adatbázisból dolgozik, hanem dinamikusan tud új látványosságokat keresni és térképen megjeleníthető útvonalat készíteni.

A backend főbb komponensei:

- UsersController: felhasználók létrehozása, listázása és preferenciák kezelése.
- PoisController: POI-k lekérdezése, környékbeli keresése és automatikus importálása.
- ItinerariesController: útvonalak generálása, mentése, lekérdezése és GeoJSON előállítása.
- ReviewsController: POI-értékelések beküldése, listázása és összesítése.
- AiScoringService: POI-k pontozása távolság, prioritás, értékelés és preferenciák alapján.
- OpenRouteService: valós útvonalgeometria lekérése a térképes megjelenítéshez.
- OverpassService: külső POI-adatok lekérése OpenStreetMap/Overpass alapján.

A backend jelenleg a következő útvonaltervezési és rangsorolási megoldásokat használja:

4.2.1 Greedy távolságalapú módszer (Nearest Neighbor)

A távolságalapú greedy módszer lényege, hogy a rendszer minden lépésben az aktuális pozícióhoz legközelebb található, még meg nem látogatott POI-t választja következő állomásként. A folyamat a kiindulási pontból indul, majd addig ismétlődik, amíg az útvonalba bevont összes látványosság sorra nem kerül.

A módszer előnye, hogy gyorsan fut és egyszerűen implementálható, ezért jól használható kisebb POI-halmazok esetén vagy összehasonlítási alapként más optimalizálási módszerek mellett.

Hátránya, hogy lokális döntéseket hoz, vagyis mindig csak a következő legközelebbi pontot vizsgálja, ezért nem garantálja a teljes útvonal globálisan optimális sorrendjét.

4.2.2 Greedy prioritásalapú rendezés

A POI-k a hozzájuk rendelt prioritási érték alapján kerülnek sorrendbe. Ez akkor hasznos, amikor a felhasználó a „legfontosabb” látványosságokat szeretné előre venni.

Mixed súlyozott algoritmus

A rendszer két tényezőt együttesen értékel: a távolságot és a prioritást. Mindkét érték külön súlyt kaphat, így a felhasználó személyre szabott sorrendezést kérhet. Ez a módszer kiegyensúlyozott, és gyakran jobb eredményt ad, mint a tisztán távolságalapú megközelítés.

4.2.3 AI-alapú greedy algoritmus:

Ez a legfejlettebb jelenlegi megoldás: minden POI egy pontszámot kap a felhasználói preferenciák súlyozása alapján. A rendszer mindig azt a POI-t választja következőnek, amely a legmagasabb összesített értékelést kapja. A pontszámítás figyelembe veszi:

- a távolságot (büntető faktor),
- a POI prioritását,
- a felhasználói értékelések átlagát.

Ez egy AI-támogatott szemléletű, súlyozott döntési logika, amely a felhasználói preferenciák alapján személyre szabottabb útvonaljavaslatot eredményez.

4.2.4 Automatikus POI-kiválasztás sugár alapján (AI Discover Mode)

A rendszer a Haversine-formulával kiszámítja, mely POI-k esnek a felhasználó által megadott körzeten belül. A jelöltek ezután AI pontszámot kapnak, és a rendszer automatikusan kiválasztja a legígéretesebb helyszíneket. Ez teszi lehetővé, hogy a felhasználónak ne is kelljen POI-kat választania: az alkalmazás maga javasol egy programot.

A backend tehát nemcsak adatokat kezel, hanem valódi döntéstámogató rétegként működik: a felhasználói preferenciákat, az értékeléseket és a földrajzi tényezőket együtt veszi figyelembe. A jelenlegi implementáció stabil, gyorsan fut, és kiváló alapot biztosít a komplexebb algoritmusok későbbi integrációjához. A backend felépítése a jövőbeli fejlesztésekhez is megfelelő alapot biztosít. A moduláris felépítés lehetővé teszi további útvonaltervező és rangsorolási módszerek későbbi beépítését. A rendszer jelenleg OpenRouteService segítségével kér le valós útvonalgeometriát, későbbi fejlesztésként pedig további szolgáltatások, például OSRM, Google Directions API vagy tömegközlekedési adatforrások is integrálhatók. Hasonló módon az AI-funkciók továbbfejleszthetők ML.NET vagy Python-alapú modellekkel, amelyek a későbbiekben képesek lehetnek valós felhasználói viselkedés alapján adaptálni az útvonaltervezési preferenciákat.

Összességében a backend egy stabil, könnyen bővíthető és tiszta architektúrájú API-réteg, amely minden frontenden képes egységes és gyors kiszolgálást nyújtani. A jelenlegi implementáció már teljeskörűen támogatja az útvonalgenerálást, a POI-kezelést, az értékelési rendszert és a felhasználói preferenciák kezelését, így a projekt további fejlesztési fázisai számára is megbízható alapot biztosít.

4.3 Felhasznált külső API-k és szolgáltatások

A TripPlanner rendszer több külső API-t és szolgáltatást használ annak érdekében, hogy a felhasználó által megadott címekből útvonaltervezésre alkalmas adatokat állítson elő, automatikusan látványosságokat keressen, AI-alapú kiegészítéseket készítsen, valamint térképen megjeleníthető útvonalat hozzon létre. A külső szolgáltatások használata lehetővé teszi, hogy az alkalmazás ne kizárólag statikus, előre feltöltött adatbázisból dolgozzon, hanem dinamikusan tudjon reagálni a felhasználó által megadott helyszínre.

4.3.1 Nominatim API – geokódolás

A felhasználó az alkalmazásban nem koordinátákat ad meg, hanem természetes nyelvű címet vagy helyszínnevet. A megadott cím koordinátává alakításához a rendszer a Nominatim szolgáltatást használja, amely az OpenStreetMap adatai alapján végez geokódolást.

A geokódolás során a rendszer a felhasználó által megadott szöveges címből szélességi és hosszúsági koordinátát állít elő. Ezek a koordináták képezik az útvonaltervezés alapját, mivel ezek alapján történik a környékbeli POI-k keresése, a távolságszámítás, valamint a térképes megjelenítés.

Ez a megoldás felhasználóbarátabbá teszi az alkalmazást, mert a felhasználónak nem kell manuálisan koordinátákat megadnia, elegendő egy ismert helyszínt vagy címet beírnia.

4.3.2 Overpass API – automatikus POI-importálás

Az automatikus POI-importáláshoz a rendszer az Overpass API-t használja. Az Overpass API az OpenStreetMap adatbázisából képes különböző típusú helyszíneket lekérni, például turisztikai látványosságokat, történelmi helyeket, múzeumokat, kilátópontokat, galériákat, emlékműveket vagy romokat.

A rendszer a felhasználó által megadott kiindulási koordináta környezetében keres POI-kat. A lekért helyszíneket feldolgozza, majd összeveti a már meglévő adatbázisban szereplő POI-kkal. Ezzel elkerülhető, hogy ugyanaz a látványosság többször is bekerüljön a rendszerbe.

Az Overpass API használata különösen fontos a kisebb települések támogatása miatt. Segítségével az alkalmazás olyan helyeken is képes látványosságokat keresni, ahol előre nem áll rendelkezésre saját POI-adatbázis.

4.3.3 OpenRouteService – valós útvonalgeometria

A TripPlanner az útvonalak térképes megjelenítéséhez OpenRouteService integrációt használ. Ennek célja, hogy a térképen ne csupán a POI-k légvonalas összekötése jelenjen meg, hanem valós úthálózaton alapuló útvonal.

A rendszer a kiválasztott POI-k koordinátáit elküldi az OpenRouteService szolgáltatásnak, amely a megadott közlekedési mód alapján visszaadja az útvonal geometriáját. Gyalogos közlekedés esetén gyalogos útvonal, autós közlekedés esetén autós útvonal kérhető le.

A visszakapott útvonal GeoJSON formátumban jeleníthető meg a térképen. Ez pontosabb és életszerűbb megjelenítést biztosít, mivel a felhasználó olyan útvonalat lát, amely ténylegesen követi az úthálózatot.

Amennyiben az OpenRouteService nem érhető el vagy nem ad vissza megfelelő eredményt, a rendszer tartalékmegoldásként légvonalas útvonalat jelenít meg, így a térképes funkció külső szolgáltatás hibája esetén is működőképes marad.

4.3.4 OpenStreetMap és Leaflet – térképes megjelenítés

A térképes felület megjelenítéséhez a rendszer Leaflet.js könyvtárat használ. A Leaflet egy nyílt forráskódú JavaScript térképkönyvtár, amely egyszerűen integrálható HTML és JavaScript alapú webes alkalmazásokba.

A térképen megjelenő alaptérkép OpenStreetMap tile rétegekből épül fel. Erre kerülnek rá az alkalmazás saját adatai, például a kiindulási pont, az útvonalba rendezett POI-k, valamint az útvonal vonala.

A POI-k sorszámozva jelennek meg, így a felhasználó könnyen követheti a javasolt bejárási sorrendet. A látványosságokhoz popup ablakok tartoznak, amelyek megjelenítik a POI nevét, leírását, értékeléseit, valamint az értékelési oldalra mutató hivatkozást.

4.3.5 OpenAI API – AI-alapú POI-kiegészítés

A rendszer AI-alapú kiegészítő funkcióihoz OpenAI API-t használ. Az OpenAI API lehetővé teszi, hogy a rendszer nagy nyelvi modellt hívjon meg szöveges feladatok megoldására, például leírások generálására, adatok kiegészítésére vagy strukturált szöveges válaszok előállítására. Az OpenAI dokumentációja szerint a modellek képesek természetes nyelvű szövegek, strukturált adatok és egyéb szöveges válaszok generálására API-híváson keresztül.

A TripPlanner rendszerben az OpenAI API elsősorban az automatikusan importált POI-k feldolgozásánál kap szerepet. Az Overpass API-ból érkező helyszínek sok esetben csak alapadatokat tartalmaznak, például nevet, koordinátát és kategóriát. Az AI-kiegészítés célja, hogy ezekből a nyers adatokból a rendszer felhasználóbarátabb POI-adatokat hozzon létre, például rövid leírást, turisztikai jelentőséget vagy kezdeti prioritási értéket.

Az OpenAI API használata segíti, hogy az automatikusan importált látványosságok ne csak technikai adatként jelenjenek meg az alkalmazásban, hanem a felhasználó számára is értelmezhető, informatív formában. Ez különösen hasznos kisebb települések vagy kevésbé ismert helyszínek esetén, ahol nem mindig áll rendelkezésre részletes, előre elkészített turisztikai leírás.

Az API-hívások biztonsága érdekében az OpenAI API-kulcsot nem célszerű közvetlenül a frontendben vagy nyilvános forráskódban tárolni. Az OpenAI dokumentációja szerint az API-kulcs titkos adat, amelyet nem szabad kliensoldali kódban vagy publikus repositoryban megjeleníteni; helyette szerveroldalon, például környezeti változóban vagy titkos kulcskezelő szolgáltatásban érdemes tárolni.

Az értékelések generálásához az alábbi prontot használom:

Turisztikai szakértő vagy. Itt egy lista látványosságokról: {namesList}.

Értékeld mindegyiket 1-től 5-ig (priority) turisztikai fontosság alapján, és adj hozzájuk egy 1 mondatos magyar nyelvű leírást.

FIGYELEM: Ha egy név alapján az adott hely egy hétköznapi cég (pl. Kft., gyár, hulladékfeldolgozó), kisállatpanzió, bolt, vagy egyáltalán nem turisztikai célpont, akkor a priority értéke kötelezően 0 legyen!

Továbbá generálj a valódi turisztikai helyekhez 5-10 darab változatos, valósághű magyar nyelvű turistavéleményt (3-5 csillagos ratinggel)!

A válaszod SZIGORÚAN egy JSON tömb legyen a következő formátumban:

```
[
  {{
    ""name"": ""Név"",
    ""priority"": 4,
    ""description"": ""Rövid leírás"",
    ""reviews"": [
      {{ ""rating"": 5, ""comment"": ""Gyönyörű hely!"" }}
    ]
  }}
]
```

Ne írj semmi mást a JSON-ön kívül!";

Az OpenAI API válasza alapján a rendszer az automatikusan importált turisztikai látványosságokhoz kezdeti értékeléseket is rendelhet. Ennek célja, hogy az alkalmazás kezdeti használati szakaszában is legyen viszonyítási alap arra, hogy egy adott POI mennyire lehet érdekes vagy ajánlott a felhasználók számára.

Ez a megoldás főként a rendszer induló állapotában hasznos, amikor még kevés valódi felhasználói értékelés áll rendelkezésre. Amennyiben az alkalmazást később nagyobb felhasználói kör használja, a mesterségesen generált kezdeti értékelések szerepe csökkenthető vagy teljesen elhagyható, mivel a rendszer ekkor már a valós felhasználói visszajelzések alapján tudja rangsorolni a látványosságokat.

4.4 Adatbázis

A rendszer adatkezelése Entity Framework Core segítségével történik. A fejlesztési környezetben az adatbázis relációs szemléletű struktúrát követ, amelyben a főbb entitások a felhasználók, a látványosságok, az útvonaltervek, az értékelések és a felhasználói preferenciák.

A felhasználók adatai a Users táblában kerülnek tárolásra. Ez tartalmazza a felhasználó azonosítóját, nevét, e-mail-címét, valamint a későbbi hitelesítési és személyre szabási funkciókhoz szükséges adatokat. A pilot rendszerben a felhasználókezelés egyszerűsített formában működik, azonban a struktúra később bővíthető teljes hitelesítési megoldással.

A POI tábla tartalmazza a látványosságok adatait, például a nevet, a koordinátákat, a leírást és a prioritási értéket. Ezek az adatok képezik az útvonaltervezés alapját, mivel a rendszer ezekből választja ki és rendezi sorrendbe a felhasználó számára releváns helyszíneket.

Az útvonaltervek tárolására külön entitás szolgál. Itt kerül eltárolásra, hogy melyik felhasználó hozta létre az adott útvonalat, milyen POI-k szerepelnek benne, milyen sorrendben kerültek bejárásra, valamint milyen optimalizálási mód alapján jött létre az útvonal.

Az értékelések külön táblában kapnak helyet. Egy értékelés kapcsolódik egy felhasználóhoz és egy POI-hoz, valamint tartalmazza a pontszámot és opcionálisan a szöveges véleményt. Az értékelések alapján a rendszer átlagos POI-minősítést számolhat, amely később az útvonaltervezési pontozásban is szerepet kaphat.

A felhasználói preferenciák tárolása azért szükséges, mert a rendszer személyre szabott útvonaljavaslatokat készít. A preferenciák között szerepelhet például a távolság, a prioritás, az értékelés és a maximális POI-szám súlya. Ezek alapján a rendszer a különböző felhasználók számára eltérő útvonalakat is ajánlhat.

5. RÉSZLETES SPECIFIKÁCIÓ

5.1 Felhasználókezelés

A rendszer felhasználóalapú működésének célja az útvonalak és AI-beállítások személyre szabása. A pilot verzió egyszerű, név alapú bejelentkezést alkalmaz:

- A felhasználó egy nevet ad meg a kezdőlapon.
- Az API a UserController segítségével létrehoz egy új rekordot, vagy visszaadja a már létezőt.
- A frontenden a felhasználó azonosítója (ID) és neve localStorage-ben kerül tárolásra.
- A későbbi funkciók (útvonalak mentése, preferenciák tárolása, értékelések beküldése) mind ehhez az ID-hez kapcsolódnak.

A felhasználókezelés egyszerűsített, névalapú működésre épül. A felhasználó a kezdőoldalon megad egy nevet, amely alapján a rendszer létrehoz egy új profilt, vagy visszaadja a már létező felhasználót. A frontend a felhasználó azonosítóját és nevét a böngésző local Storage tárhelyében tárolja. Ennek köszönhetően a különböző oldalak ugyanahhoz a felhasználóhoz tudják kapcsolni az útvonalakat, preferenciákat és értékeléseket. Éles környezetben ez a megoldás kiegészíthető lenne biztonságosabb hitelesítéssel, például JWT- vagy OAuth-alapú autentikációval.

5.2 Látványosságok kezelése

A látványosságokat a rendszer egy JSON forrásból importálja (Budapest POI-k), majd az alkalmazás indításakor az adatbázisban tárolja őket. Egy POI tartalmazza:

- azonosító
- név
- koordináták (lat, lng)
- prioritás mező (future AI modellhez)
- rövid leírás vagy metaadat

Az API a következő műveleteket támogatja:

- GET /api/pois – POI-k listázása
- GET /api/pois/{id} – adott POI lekérdezése
- GET /api/pois/{id}/rating-summary – átlagértékelés, darabszám
- GET /api/pois/{id}/reviews – értékelések lekérdezése

A frontend a new-itinerary oldalon és a map oldalon is valós időben jeleníti meg a POI-k értékelését.

A POI-kezelés nemcsak az adatbázisban szereplő látványosságok listázását jelenti, hanem a környékbeli helyszínek keresését és új POI-k automatikus importálását is. A rendszer a megadott koordináta alapján képes lekérdezni, hogy mely POI-k találhatók a felhasználó közelében.

Ha kevés látványosság áll rendelkezésre, a felhasználó elindíthatja a környék felfedezését. Ilyenkor a rendszer Overpass API segítségével új helyszíneket keres, majd azokat eltárolja az adatbázisban.

5.3 Útvonal generálása

Az alkalmazás egyik központi funkciója az útvonal generálása, amely a felhasználó által megadott kiindulási pontból indul ki. A felhasználó első lépésben egy címet vagy helyszínnevet ad meg, amelyet a rendszer a Nominatim OpenStreetMap API segítségével koordinátává alakít. A kapott szélességi és hosszúsági értékek alapján történik a környékbeli POI-k keresése és az útvonal előállítása.

A felhasználó megadhatja a keresési sugarat, a maximálisan bevonni kívánt POI-k számát, valamint a közlekedési módot. A radiusKm érték határozza meg, hogy a rendszer hány kilométeres körzetben keressen látványosságokat, a maxPois azt adja meg, hogy legfeljebb hány POI kerüljön az útvonalba, míg a travelMode a közlekedési módot jelöli, például gyalogos vagy autós útvonalat.

Az útvonaltervezés kétféle módon történhet. Kézi útvonaltervezés esetén a felhasználó maga választja ki a megjelenített POI-listából azokat a látványosságokat, amelyeket szeretne meglátogatni. A kiválasztott POI-k sorrendjét a rendszer genetikus algoritmus segítségével optimalizálja, amely több lehetséges bejárási sorrendet vizsgál, és ezek közül választja ki a legkedvezőbbet.

Automatikus útvonaltervezés esetén a felhasználónak nem kell kézzel kiválasztania a látványosságokat. A rendszer a megadott keresési sugáron belül található POI-kat vizsgálja, majd a távolság, a prioritás, az értékelések és a felhasználói preferenciák alapján kiválasztja a leginkább ajánlott helyszíneket. Ezután az útvonaloptimalizáló logika meghatározza ezek bejárási sorrendjét.

A rendszer a POI-k szűréséhez és az útvonalváltozatok gyors kiértékeléséhez Haversine-alapú távolságszámítást használ. Ez lehetővé teszi, hogy a backend meghatározza, mely látványosságok esnek a megadott körzeten belül, illetve, hogy az optimalizálás során gyorsan összehasonlíthatók legyenek a különböző útvonalváltozatok.

Az elkészült útvonal mentésre kerül az adatbázisba a felhasználóhoz kapcsolva. A rendszer eltárolja a kiindulási pontot, a kiválasztott POI-kat, az optimalizált sorrendet és a közlekedési módot. Az útvonalhoz GeoJSON formátumú térképi adat is készül, amely alapján a létrehozott útvonal a térképes felületen megjeleníthető. A mentett útvonalak később az előzmények között visszakereshetők és újra megnyithatók.

5.4 Értékelési rendszer

Az értékelések az ajánlórendszer működésében is szerepet kapnak. A rendszer a POI-k átlagos értékelését és az értékelések számát is figyelembe veheti a pontozás során, így a felhasználói visszajelzések hatással lehetnek a későbbi útvonaljavaslatokra.

Főbb funkciók:

- Felhasználók 1–5 csillagos értékelést adhatnak.
- Szöveges értékelés lehetősége
- A ReviewsController kezeli a beküldést és a lekérdezést.
- A rating-summary végpont aggregált adatot ad vissza (átlag, darabszám).
- A rendszer kiszámolja az átlagot minden POI-hoz és egyetlen query-vel adja vissza.

A frontend a map popupban és a POI-listában is megjeleníti az értékelést.

5.5 Térképes megjelenítés

A TripPlanner térképes modulja a Leaflet.js könyvtárra épül, amely lehetővé teszi az útvonalak interaktív megjelenítését a felhasználói felületen. A rendszer dinamikusan betölti az adott útvonalhoz tartozó GeoJSON adatokat, majd ezek alapján megjeleníti a térképen a kiindulási pontot, az útvonalba rendezett POI-kat, valamint az útvonal vonalát.

A látványosságok sor számozva jelennek meg a térképen, így a felhasználó könnyen követni tudja a javasolt bejárási sorrendet. Az egyes POI-khoz popup ablakok is tartoznak, amelyekben megjelenik a látványosság neve, az adott helyhez kapcsolódó értékelés, valamint egy hivatkozás az értékelési oldalra. Ezáltal a térképes felület nemcsak az útvonal vizuális követését támogatja, hanem további információkat is biztosít a kiválasztott látványosságokról. A térképes megjelenítés során az alkalmazás lekéri az adott útvonal GeoJSON adatait. A térképen megjelenik az indulási pont, az útvonalba rendezett POI-k és az útvonal vonala.

A POI-khoz popup ablakok tartoznak, amelyekben megjelenik a látványosság neve, leírása és értékelései. A popupból elérhető az adott POI értékelési oldala is.

A rendszer OpenRouteService segítségével valós útvonalgeometriát is képes megjeleníteni, de szükség esetén légvonalas útvonalat rajzol.

5.5.1 OpenRouteService szerepe az útvonalmegjelenítésben

A TripPlanner rendszer az OpenRouteService segítségével képes valós útvonalgeometriát lekérni. Ez azt jelenti, hogy a térképen megjelenő útvonal nem egyszerűen a POI-k légvonalas összekötése, hanem a választott közlekedési mód alapján ténylegesen bejárható útvonal.

A rendszer a kiválasztott POI-k koordinátáit elküldi az OpenRouteService iránykereső szolgáltatásának. A szolgáltatás a megadott közlekedési mód alapján visszaadja az útvonal geometriáját. Gyalogos közlekedés esetén gyalogos útvonal, autós közlekedés esetén autós útvonal kérhető le.

A visszakapott geometria GeoJSON formátumban jeleníthető meg a Leaflet térképen. Ez pontosabb és felhasználóbarátabb megjelenítést biztosít, mivel a felhasználó nemcsak azt látja, hogy mely pontokat kell bejárnia, hanem azt is, hogy milyen útvonalon érdemes közlekednie.

A rendszer hibatűrő módon működik: ha az OpenRouteService nem érhető el, vagy nem ad vissza érvényes útvonalat, az alkalmazás légvonalas útvonalat jelenít meg. Így a térképes funkció külső szolgáltatás hibája esetén is használható marad.

Útvonaltervezőalgoritmusok összefoglalása

A turisztikai útvonaltervezés többféle algoritmikus megközelítést igényelhet. A Dijkstra és az A* algoritmus klasszikus gráfalapú legrövidebbút-kereső módszerek, amelyek főként akkor hasznosak, ha a rendszer saját úthálózati gráffal dolgozik. Ezek pontos útvonalak meghatározására alkalmasak, azonban nagyobb gráfok esetén jelentős számítási kapacitást igényelhetnek.

A genetikus algoritmus ezzel szemben nem egyetlen kiindulási és célpont közötti legrövidebb utat keres, hanem több látványosság optimális sorrendjét próbálja meghatározni. Ez jobban illeszkedik a TripPlanner céljához, mivel a felhasználó általában nem egyetlen helyre szeretne eljutni, hanem több POI-t szeretne hatékony sorrendben meglátogatni.

A Haversine-formula a rendszerben gyors távolságbecslési eszközként működik, amely támogatja a POI-k szűrését és az algoritmusok kiértékelését. A valós útvonalmegjelenítéshez az alkalmazás OpenRouteService-t használ, amely úthálózaton alapuló útvonalat biztosít.

A projektben ezek a megközelítések egymást kiegészítve jelennek meg. A Haversine-formula gyors belső számításokat biztosít, a genetikus algoritmus a POI-k sorrendjét optimalizálja, az OpenRouteService pedig a térképes megjelenítéshez valós útvonalgeometriát szolgáltat. Ez a kombináció lehetővé teszi, hogy a rendszer egyszerre legyen gyors, rugalmas és a felhasználó számára jól értelmezhető.

. Külső API-k integrációja

A rendszer több külső szolgáltatást is használ. A címek koordinátára alakítása geokódolással történik, amely lehetővé teszi, hogy a felhasználó ne szélességi és hosszúsági koordinátákat adjon meg, hanem természetes formában írjon be egy címet vagy helyszínnevet.

Az automatikus POI-importálás az Overpass API segítségével működik. Ez az OpenStreetMap adatbázisából kér le turisztikai és történelmi helyszíneket a megadott koordináta környezetéből.

A valós útvonalgeometria lekéréséhez a rendszer OpenRouteService integrációt használ. Ez lehetővé teszi, hogy a térképen ne egyszerű légvonalas összekötés jelenjen meg, hanem gyalogos vagy autós úthálózaton alapuló útvonal.

A külső API-k használata mellett a rendszer tartalékmegoldásokat is alkalmaz. Ha egy külső szolgáltatás nem érhető el, a rendszer továbbra is képes alapvető működésre, például légvonalas útvonal megjelenítésére.

5.5.2 Valós útvonalgeometria lekérése

A rendszer OpenRouteService segítségével képes valós úthálózaton alapuló útvonalgeometriát lekérni. Az útvonal pontjainak koordinátáit a backend elküldi a külső szolgáltatásnak, amely a választott közlekedési mód alapján visszaadja a bejárható útvonalat.

A visszakapott geometria GeoJSON formátumban kerül felhasználásra, így közvetlenül megjeleníthető a Leaflet térképen. Ez pontosabb és életszerűbb útvonalmegjelenítést biztosít, mint a pontok egyszerű légvonalbeli összekötése.

A funkció hibatűrő módon működik: ha a külső útvonaltervező szolgáltatás nem érhető el, a rendszer továbbra is képes légvonalas útvonalat megjeleníteni.

5.6 Automatikus POI-importálás

Az automatikus POI-importálás célja, hogy a rendszer képes legyen új látványosságokat felvenni a felhasználó által megadott hely környezetéből. A funkció az új útvonal létrehozása oldalon érhető el a „Környék felfedezése AI-val” gombbal.

A rendszer először ellenőrzi, hogy rendelkezésre állnak-e az indulási koordináták. Ezután Overpass API segítségével lekéri a környékbeli turisztikai és történelmi helyszíneket. A találatokat összeveti a meglévő POI-adatokkal, majd az új helyszíneket eltárolja az adatbázisban.

Az importálás után a frissen bekerült POI-k megjelenhetnek a választható látványosságok listájában, és felhasználhatók útvonaltervezéshez

5.7 AI preferenciák

A TripPlanner rendszer felhasználói szinten konfigurálhatóvá teszi az útvonaltervezés működését különböző súlyparaméterek segítségével. A felhasználó ezeket az AI beállítások menüpontban adhatja meg:

- WeightDistance – mennyire büntesse a rendszer a hosszabb távolságot,
- WeightPriority – mennyire preferálja a magas prioritású POI-kat,
- WeightRating – mennyire részesítse előnyben a magas átlagértékelésű helyeket,
- MaxPois – legfeljebb hány POI szerepelhet egy útvonalban.

A súlyok az alábbi területeken jelennek meg:

1. POI-k kiválasztása
   A rendszer a radius-on belül található POI-kat értékeli három tényező alapján:
   - távolság a kezdőponttól (WeightDistance)
   - POI prioritása (WeightPriority)
   - POI átlagértékelése (WeightRating)

A végső pontszám kiszámítása egy súlyozott scoring formula alapján történik, így a rangsorolás valóban személyre szabott.

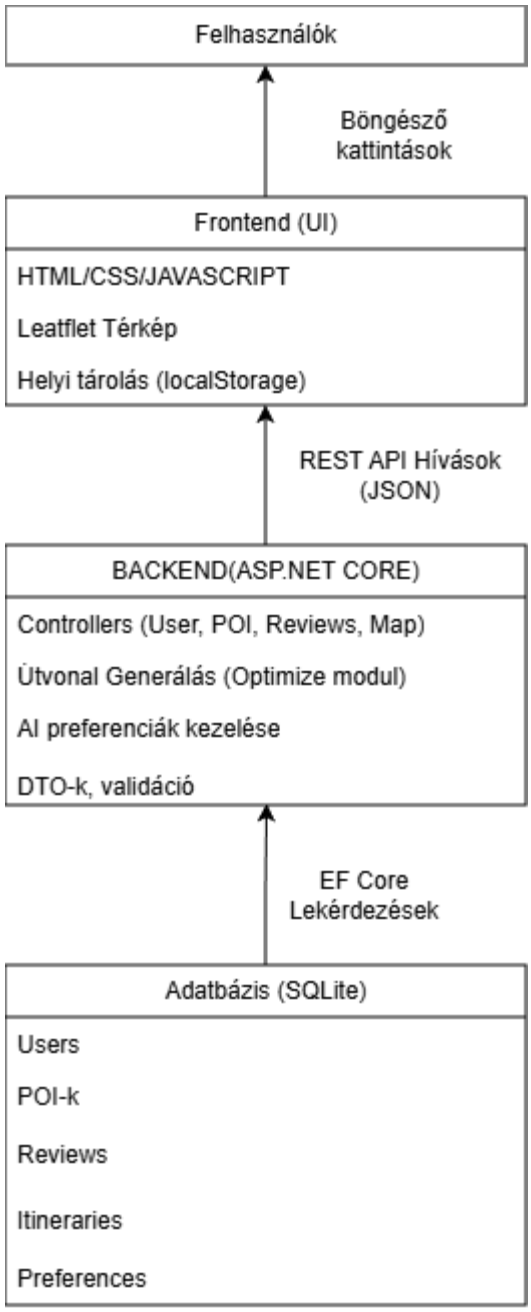
2. POI-k sorrendje az útvonalban
   Az optimalizáló modul a fenti súlyozott pontszámokat is figyelembe veszi akkor, amikor meghatározza, milyen sorrendben kerüljenek be a POI-k.
3. Különböző felhasználók különböző útvonalakat kapnak
   Mivel minden felhasználó saját AI-beállításokkal rendelkezik, ugyanarra az indulópontra különböző útvonalak jöhetnek létre.

A rendszer AI-támogatott működése súlyozott pontozási modellen alapul. A pontozás során a rendszer figyelembe veszi a távolságot, a POI prioritását, az átlagos értékelést, az értékelések számát és a felhasználói preferenciákat.

A cél az, hogy a rendszer ne általános, mindenki számára azonos útvonalat adjon, hanem a felhasználó igényeihez igazodó javaslatot készítsen.
Továbbfejlesztési lehetőségek:

- gépi tanulás alapú finomhangolással (ML.NET vagy Python modell),
- valós felhasználói viselkedés alapján adaptív súlyozással,
- dinamikus súly módosítással adott napszak, időjárás vagy tömeg alapján.

6. RENDSZER BEMUTATÁSA



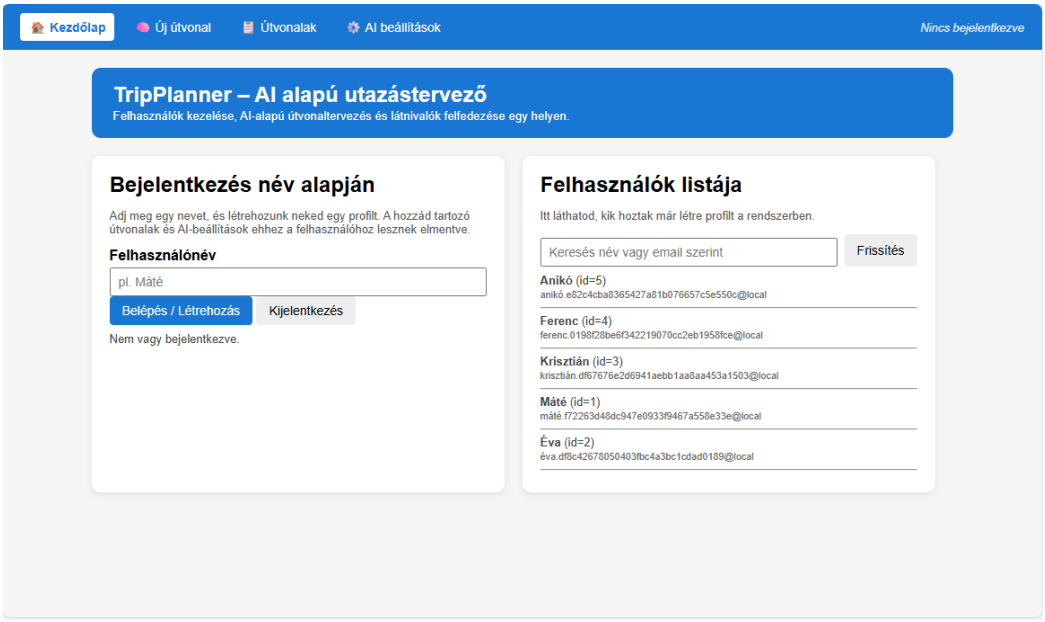
6.1 Funkciók bemutatása:

A rendszer a kiindulási pont beírása után az Overpass API-n keresztül lekéri az adott pont körülötti látványosságokat. A POI-k az importálás után elérhetővé válnak az útvonaltervezéshez valamiunt a térképes megjelenítéshez

A rendszer funkcióit jelenleg 6 html oldalon keresztül érhetjük el.

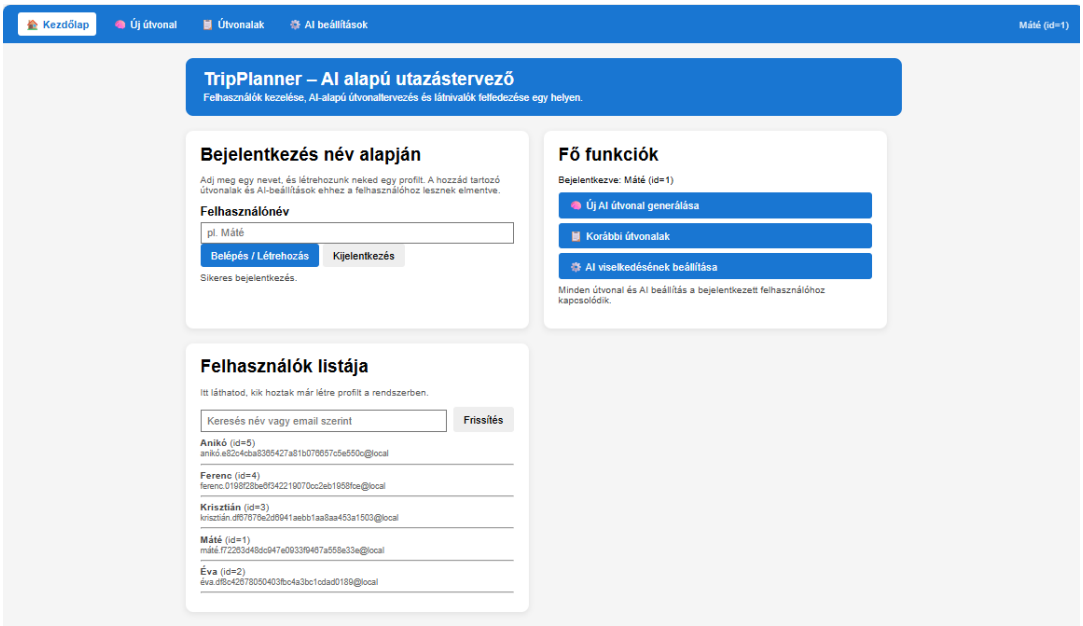
6.2 kezdő felület (index.html)

Az index.html megnyitásával az alkalmazás főoldalával találkozhatunk. Mivel az alkalmazásunk teszt fázisban van így ezen az oldalon megtalálhatjuk az adatbázisban található felhasználókat is. Minden felhasználóhoz tartozik egy UserID azonosító. Ezen az azonosítón keresztül tudjuk nyomon követni a felhasználók által létrehozott értékeléseket, létrehozott útvonalakat és elérni az egyéni beállításokat.



6.2.1 ábra index.html

A bejelentkezés után hozzá férünk az alkalmazás menüjéhez és a fő funkciók eléréséhez.

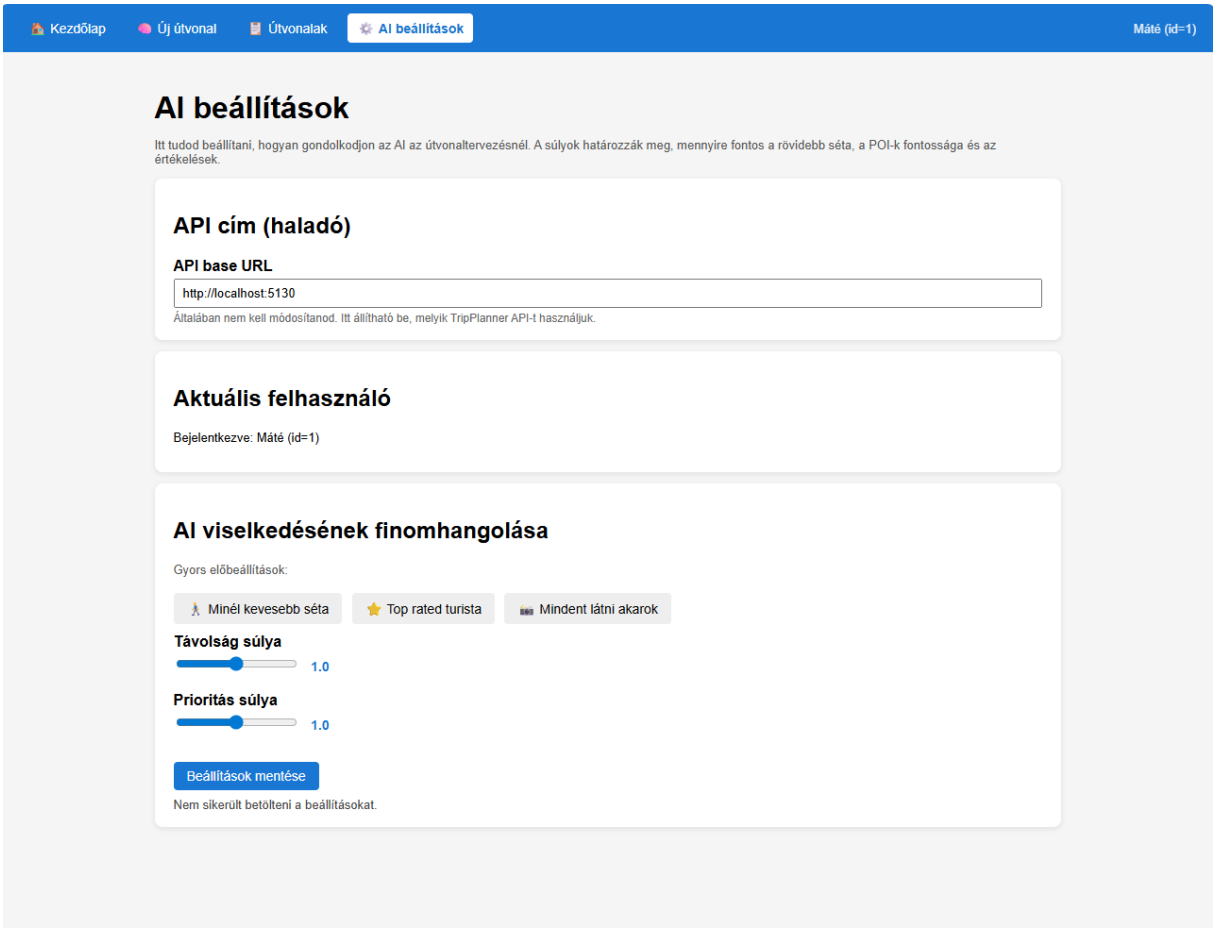


6.2.2. ábra index.html bejelentkezés után

6.3 Beállítások (User-Preferences.html)

A beállítások pontban az útvonaltervező AI viselkedését tudjuk finom hangolni az alábbi értékek módosításával

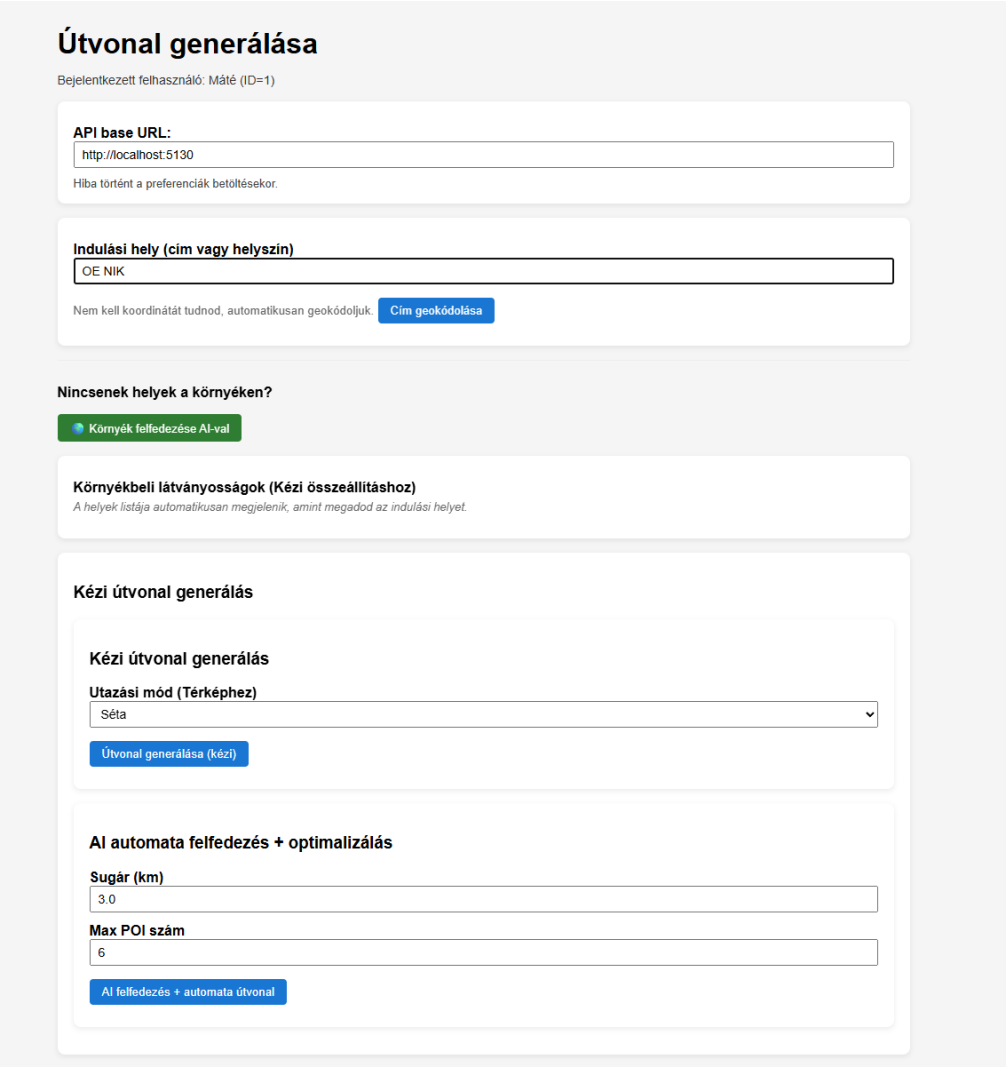
- Távolság súlya – a kiindulóponttól messzebb lévő látványosságok háttérbe szorulnak és a közelebbi látványosságok kerülnek előbb bele a rendszerbe
- Prioritás súlya - Ez a beállítás szabályozza, hogy az útvonaltervező mennyire vegye figyelembe a látványosságokhoz tartozó *prioritás* értéket.
    - Ha az érték magas, akkor a rendszer előnyben részesíti az általad fontosnak jelölt vagy kiemelt látnivalókat.
    - Ha az érték alacsony vagy 0, akkor a prioritás nem számít bele a pontszámításba, és az ajánlások inkább más tényezők — például a felhasználói értékelések és a távolság — alapján dőlnek el.



4.3.1. ábra user-preferences.html

6.5 Új útvonal (new-itinerary.html)

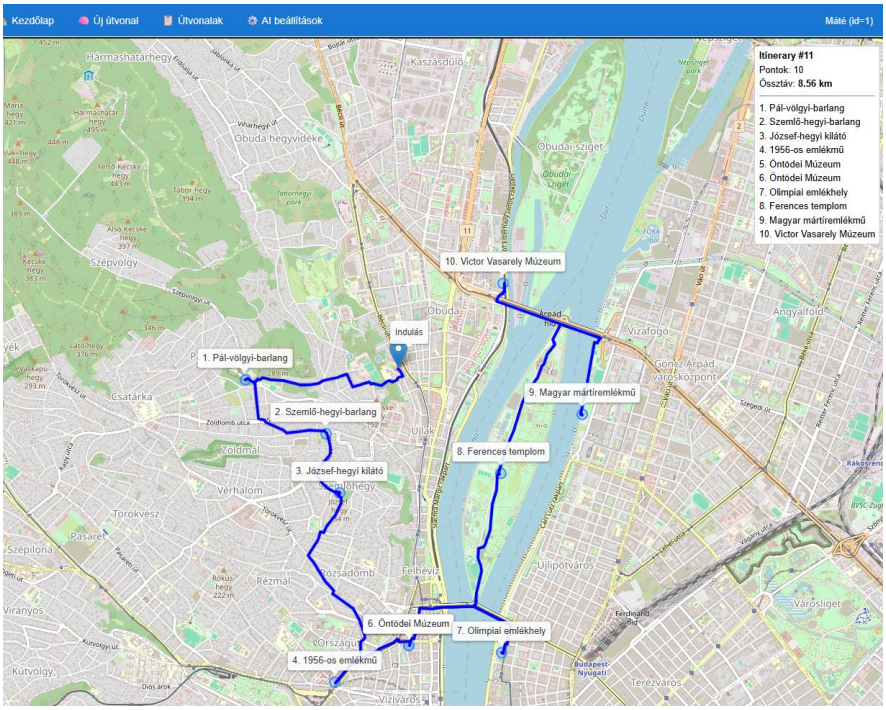
Ezen az oldalon tudjuk legenerálni az útvonalat.
Az indulási hely menüpontnál elég beírnunk az indulásunk címét vagy egy népszerű teret és miután rányomunk a geokódolásra a a frontend a Nominatim OpenStreetMap API hívásával átalakítjuk a címet koordinátákká a tesztnél az OE NIK-et használjuk kiindulási pontnak.
Jelenleg a rendszerben két féle módon lehet útvonalat tervezni.



6.4.1. ábra new-itinerary.html

6.6 Térképes megjelenítés (map.html)

Az útvonal generálása után megnyílik számunkra a térkép, ahol láthatjuk a látványosságok helyszínét és a látogatás ajánlott sorrendjét.



6.5.1. ábra map.html

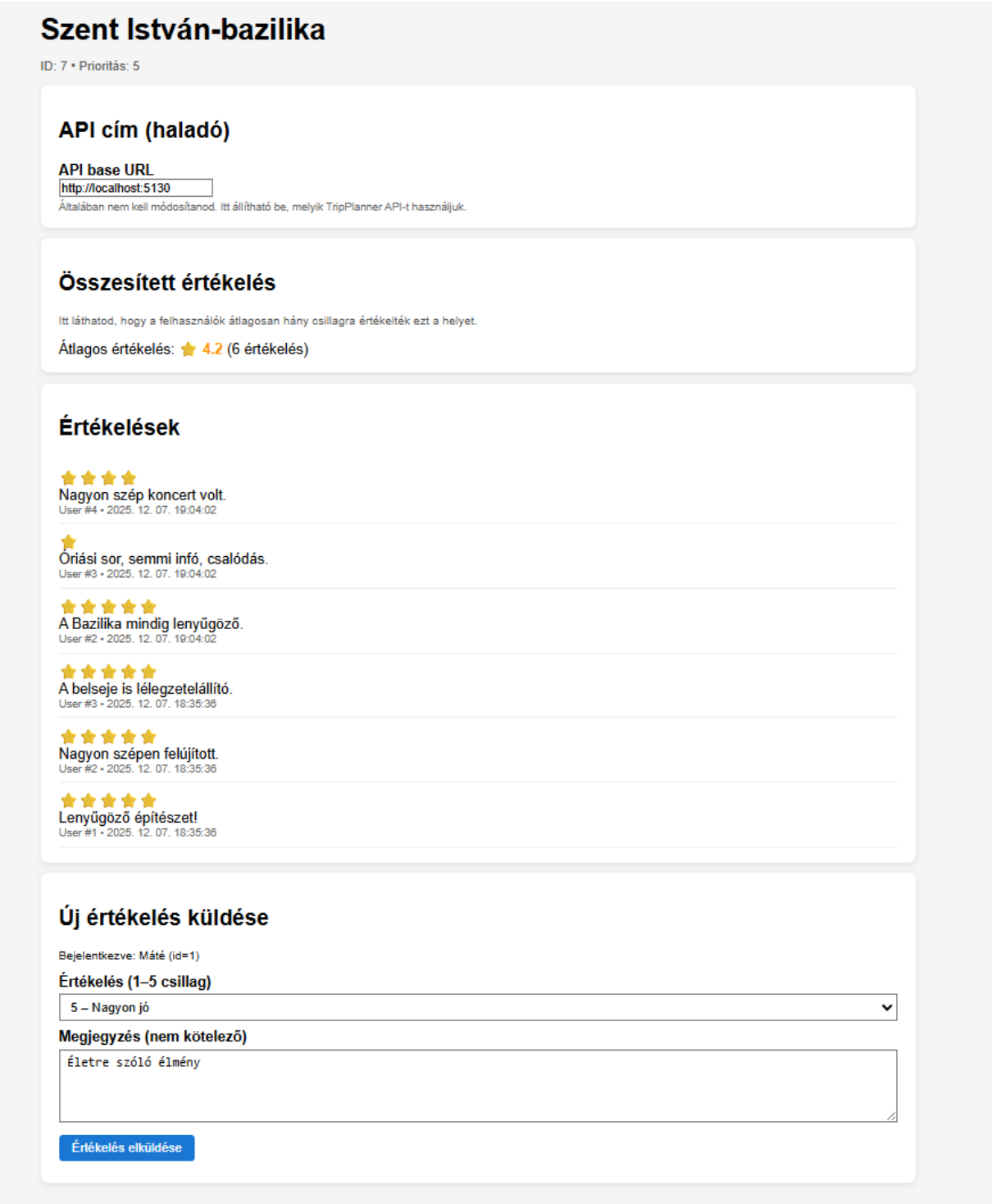
A látványosságokra rákattintva láthatjuk a felhasználók által megadott értékeléseket.



6.5.2. ábra map.html értkelés bemutatás

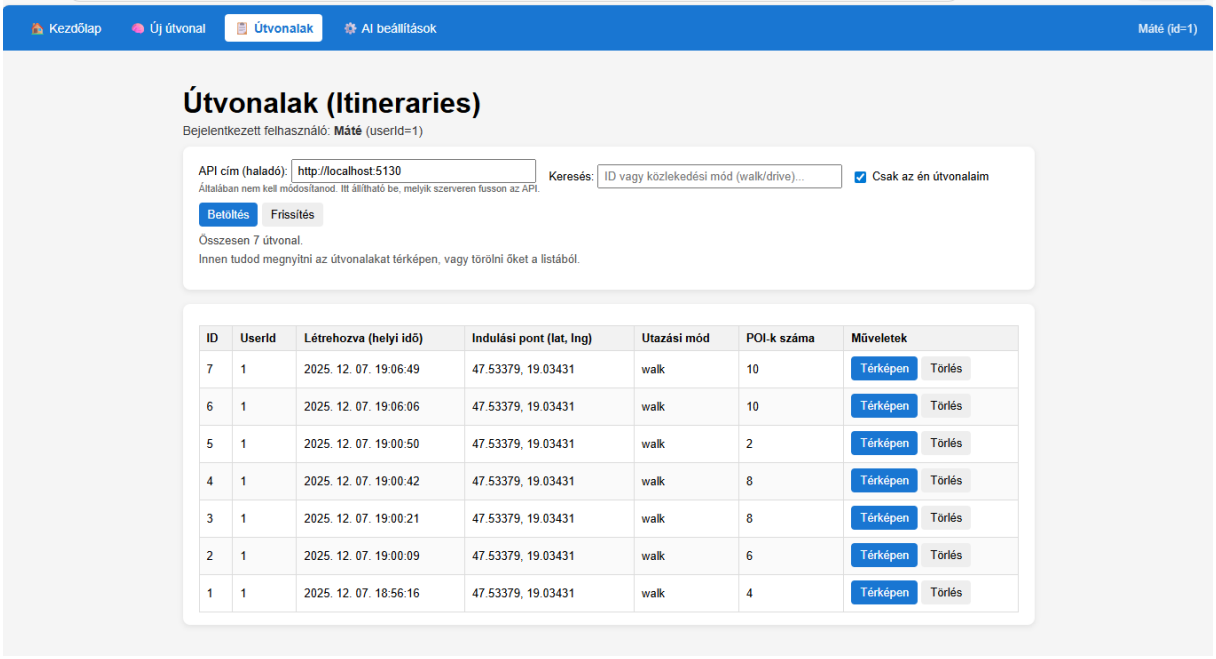
Itt tudjuk megnyitni az adott látványosság értékelési oldalát látthatjuk az előzetes értékeléseket és tudunk írni is saját értékelést. Ezek az értékelésen nagyban befolyásolják az útvonal tervezését.

6.7 Értékelések (poi-reviews.html)



6.6.1. ábra poi-reviews.html

6.8 Előzmények megtekintése (itineraries.html)



6.7.5. ábra itineraries.html

A rendszer jelenlegi állapotával be tudjuk bizonyítani, hogy el tudjuk érni a kitűzött célt. A rendszer főbb funkciói (útvonalgenerálás, POI kezelés, értékelési rendszer, térképes megjelenítés) már implementálva vannak.

7. IRODALOMJEGYZÉK

[1] „Wanderlog,” [Online]. Available: https://wanderlog.com/. [Hozzáférés dátuma: 20 április 2025].

[2] Pebblar. [Online]. Available: https://staging.pebblar.com/. [Hozzáférés dátuma: 20 Április 2025].

[3] „Sygic Travel,” [Online]. Available: https://www.sygic.com/travel. [Hozzáférés dátuma: 19 április 2025].

[4] „Rome2Rio,” [Online]. Available: https://www.rome2rio.com/. [Hozzáférés dátuma: 19 április 2025].

[5] E. W. Dijkstra, „Oral history interview with Edsger W. Dijkstra,” University of Minnesota, Minneapolis, Charles Babbage Institute, 2001.

[6] E. W. Dijkstra, „A Note on Two Problems in Connexion with Graphs,” in *Numerische Mathematik*, 1959, pp. 269-271.

[7] P. Katalin, „Legrövidebb útkereső algoritmusok diplomamunka,” ELTE TTK, Budapest, 2009.

[8] H. Zoltán, „Valós idejű adaptív A* útkeresési algoritmus,” Méréstechnika és Információs, BME VIK, 2012.

[9] H. H. &. A. B. Akylai Bolotbekova, „TTrip route optimization based on bus transit using genetic algorithm with different crossover techniques,” Nature Portfolio, Necmettin Erbakan University, 2025.

[10] P. Georgakis, A. Almohammad, E. Bothos, B. Magoutas és Arnaoutaki, „MultiModal route planning in mobility as a service,” ACM, 2019.

[11] L. H. S. H.-L. T. Z. Anne Durand, „Mobility-as-a-Service and changes,” Ministry of Infrastructure and Water Management, Hollandia, 2018.

[12] R. J. L. a. R. E. Tarjan, „SIAM J. Appl. Math,” in *A separator theorem for planar graphs,*, 1979, pp. 177-189.

[13] P. K. S. R. S. S. Monika R. Henzinger, „article no. SS971493,” in *Faster Shortest Path Algorithms for Planar Graphs, journal of computer and system sciences 55*, 1997, pp. 3-23.

[14] G. N. Frederickson, „Data Structures for OnLine Updating of Minimum Spanning Trees, with Applications,” Computer Science Technical Reports, Department of Computer Science, Purdue University, 1984, pp. 83-449.

[15] A. Ozpinar, „A Hyper-Integrated Mobility as a Service (MaaS) to Gamification and Carbon Market Enterprise Architecture Framework for Sustainable Environment,” Creative Commons Attribution, Svájc, 2023.

[16] „Google Maps Platform hivatalos dokumentáció,” Google, [Online]. Available: https://developers.google.com/maps/documentation. [Hozzáférés dátuma: 19 Április 2025].

[17] „HERE Technologies fejlesztői portál,” [Online]. Available: https://developer.here.com. [Hozzáférés dátuma: 19 április 2025].

[18] „OpenTripPlaner Github,” [Online]. Available: https://github.com/opentripplanner/OpenTripPlanner. [Hozzáférés dátuma: 19 április 2025].

[19] „OpenTripPlanner wiki,” [Online]. Available: https://www.opentripplanner.org. [Hozzáférés dátuma: 19 április 2025].

[20] „TiripIT,” [Online]. Available: https://www.tripit.com/web. [Hozzáférés dátuma: 20 április 2025].

[21] „Wingman,” [Online]. Available: https://wman.com/hu/. [Hozzáférés dátuma: 20 április 2025].

8. ÁBRAJEGYZÉK

3.1.1.1. ábra Dijkstra Algoritmus Működése 10
3.1.2.1. ábra A* Algoritmus működése 11
3.1.3.1. ábra Genetikus Algoritmus működése 12
6.3.1. ábra user-preferences.html 29
6.7.1. ábra itineraries.html 33